Circuits Updates - January 2024

We report a number of developing ideas on the Anthropic interpretability team, which might be of interest to researchers working actively in this space. Some of these are emerging strands of research where we expect to publish more on in the coming months. Others are minor points we wish to share, since we're unlikely to ever w paper about them.

We'd ask you to treat these results like those of a colleague sharing some thoughts or preliminary experiments few minutes at a lab meeting, rather than a mature paper.

Short Research Notes

- Interpretability Team Priorities
- Further investigation of attention superposition
- Can dictionary learning uncover sparse features in an MNIST model?
- Features in an 8-layer Model
- Multilingual Features in Large Models
- Ghost Grads: An improvement on resampling
- Counterexamples in Superposition
- Predicting Future Activations
- Random Open Problems

Updates

- New Comment On Previous Papers
- Research By Other Groups

Interpretability Team Priorities

Chris Olah, Shan Carter, Adam Jermyn, Joshua Batson, Tom Henighan

As our team has grown, we have gotten more bandwidth to focus on a wider range of directions. In light of thi our recent results on dictionary learning, our plan over the coming months is to focus on three areas:

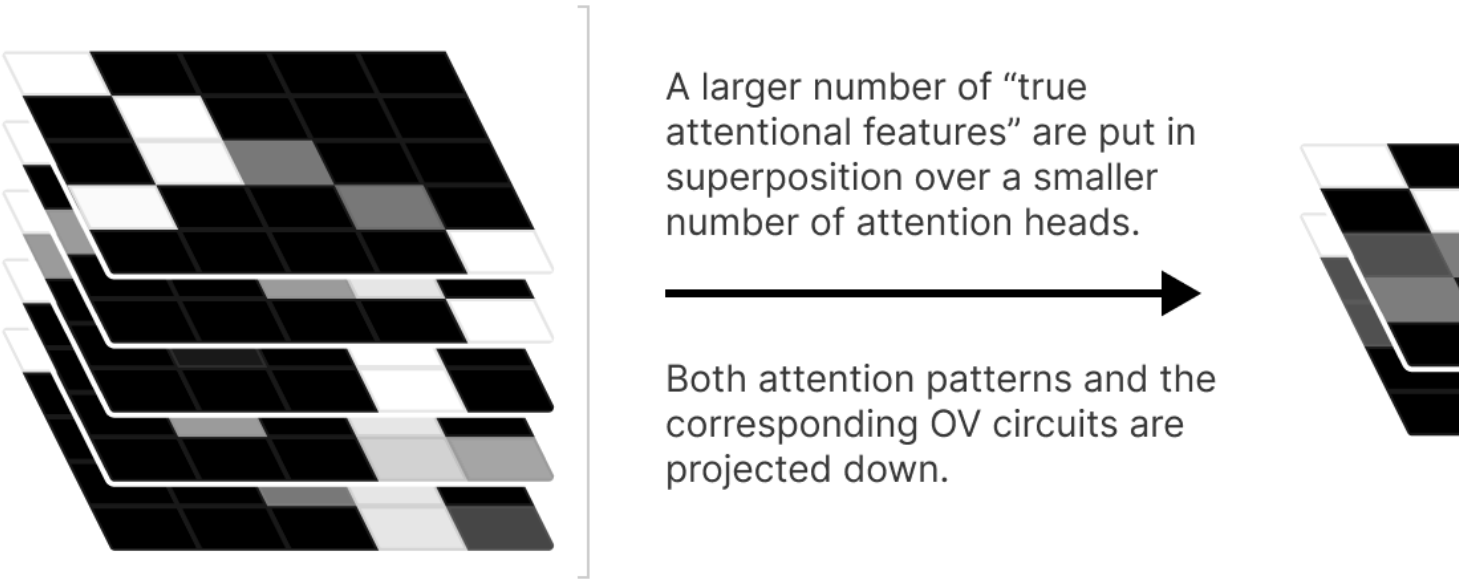
- Dictionary Learning Scaling and Science
- Attacking Attention Superposition in Real Models with Dictionary Learning
- Understanding Circuits Based on Above

Further investigation of attention superposition

Adam Jermyn, Chris Olah, Tom Conerly

In our investigations of attention superposition (see our previous discussion as well as Larson and Greenspan 8 Wynroe), one toy model we've studied has shown geometry which is strongly reminiscent of that in Toy Model Superposition.

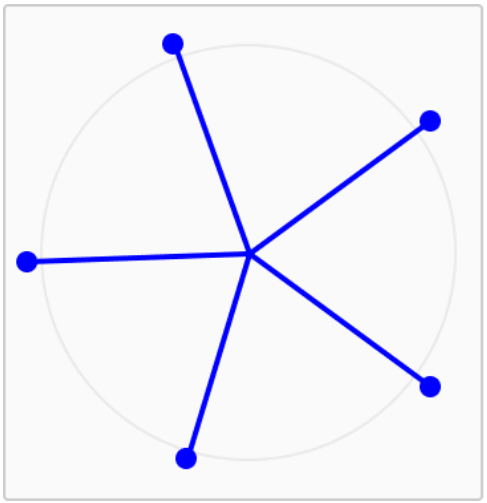
In this model, we imagine that there are several true "attentional features", each captured by a single attention with sparse attention patterns and a sparse OV circuit. We then train a model with a smaller number of attentic heads to represent these attentional features.



Experiment Details: We first generate random (fixed) OV weights for five attention heads. For each sample, we generate a residual stream across multiple tokens filled with random numbers, as well as a random attention pa for each head. The attention patterns and OV circuits are both highly sparse. The target output is the result of applying these attention heads to the residual stream, followed by a ReLU nonlinearity. This forms the training

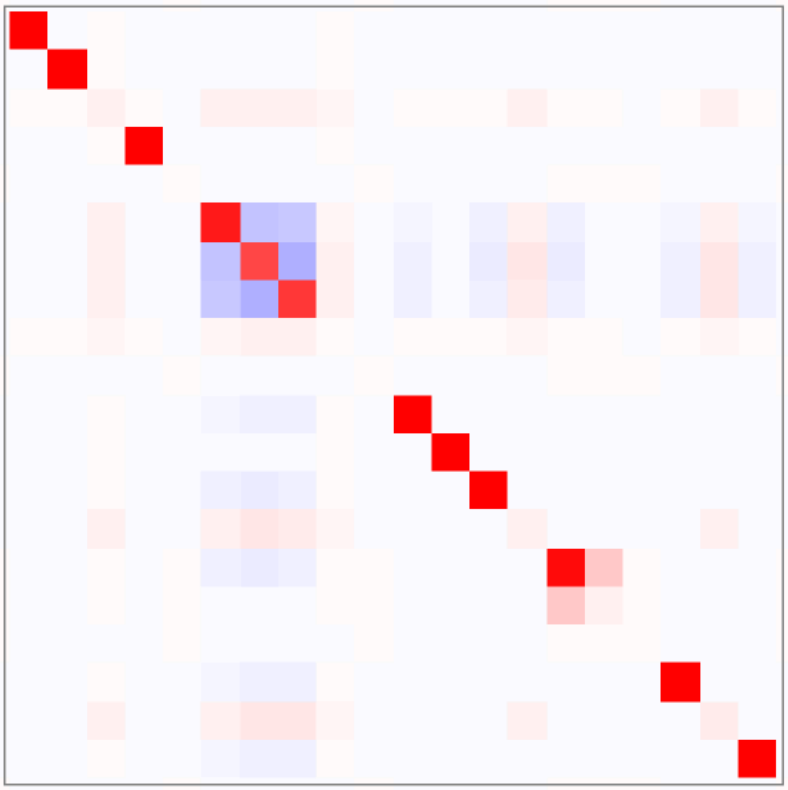
The model itself takes as input the five generated attention patterns for each sample and learns a matrix that li combines them down into just two attention heads. It uses the same mixing matrix to combine the five OV wei down into just two heads. We apply this model to the residual stream, apply a ReLU afterwards, and compute t MSE loss versus the target output.

Results: The main thing we're interested in is the geometry of how the true attentional features are organized. train a model with 5 "true attentional features" and two attention heads, we can plot the mixing matrix as a sca plot, enabling us to directly inspect the geometry. This reveals a striking geometry:

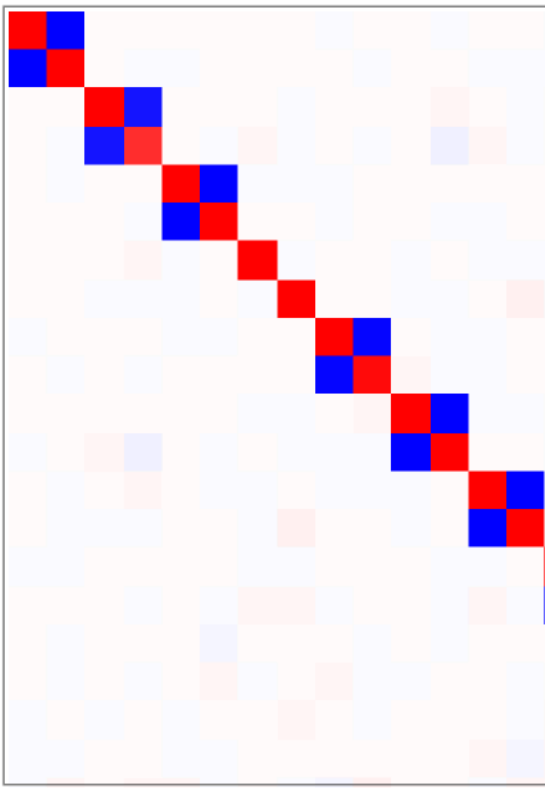


If we examine the mixing matrix, which maps attention patterns and OV circuits down to the two real attention heads, we see that it produces similar geometry to that found in “Toy Models of Superposition”.

A triangle!



Antipodes



A factor that we found important in generating superposition is ensuring that the OV circuit is sparse: when the circuit is dense we tend not to see signs of superposition.

Can dictionary learning uncover sparse features in an MNIST model?

Adam Jermyn, Adly Templeton, Joshua Batson, Trenton Bricken, Chris Olah

When we began our research into dictionary learning, we initially investigated a problem we expected to be ea than transformer language models: very simple neural networks trained to classify MNIST digits.

This problem might be seen as a kind of minimal example of a real neural network we still don't understand. To best of our knowledge, there is no mechanistic account of how even very simple neural networks classify MNIS one trains a model without a hidden layer – simply doing a generalized linear regression with a softmax on the mechanistic analysis is relatively straightforward, with each class being supported or inhibited by different pixe as soon as one adds a hidden layer, the problem becomes poorly understood. The resulting hidden layer neuro appear inscrutable.

We found this problem harder than we expected, and eventually switched to just directly attacking language m (leading to our paper). But along the way we had some interesting results, and also found some interesting challenges.

Our failure to understand MNIST models might be seen as an example of a more general trend: mechanistic interpretability has had little success in reverse engineering MLP layers in small but real neural networks, despit successes in reverse engineering portions of more sophisticated neural networks like InceptionV1 and a 1-layer transformer. One hypothesis is that these challenges have actually been due to superposition, but there might be other reasons (perhaps small models are just pathological in some way and don't have "crisp abstractions"!) Understanding this seems valuable.

Our Results So Far

In our investigations of dictionary learning, we attempted to recover sparse features from the activations of fu connected ReLU models trained on MNIST.

We performed hyperparameter scans of dictionary learning with a variety of algorithms on fully-connected MN models with 16-64 neurons per layer and 1-3 layers. The resulting features often appeared interpretable, and c mapped to individual digits. In particular, we often saw multiple features corresponding to slightly different wa drawing each digit, suggesting that the networks learned to use a template-matching strategy. These features had matching downstream effects on the logits (e.g. a “0” feature enhanced the probability placed on the digit

These features seem like one natural possibility for features the model might represent in superposition. If a m without a hidden layer is forced to have a single "general template" for each digit, having many different temp for the different ways a digit can be drawn might be a natural improvement. And because the templates are all mutually exclusive (there is only one actual digit drawn!) they are naturally sparse, which makes it possible to e them in superposition with minimal interference costs.

Another possibility could have been more composable features describing parts of a digit, such as edges, or lin loops at different positions. These are what we expect to see in early layers if we train a conv net! However, we see anything like this – we only saw features corresponding to different "templates" of digits.

We have some uncertainties about how to interpret these results. On the one hand, we saw clear signs of interpretable features in networks trained on MNIST. On the other hand, there was no clear sign that they were features", rather than just being clusters of similar data points. For example, one thing we'd hoped to find was “elbow” in the reconstruction error as a function of sparsity or number of features, but we didn't find this. More in order to achieve low reconstruction error we either needed enormous numbers of features (comparable to t dataset size) or needed to allow many different features to be active on each example. The most compelling ev for them being "real" was that the features had logit weights corresponding to the correct classes if you multip their direction vectors from the output weights (as mentioned above).

This pattern of finding interpretable features but not finding clear evidence that they're the "true features" or th we've "found all of them" continues in our new work. In some ways, this seems more expected in light of our fi about feature splitting. But an important open question remains whether we can find evidence for a "strong superposition hypothesis", or whether features are just a pragmatically useful abstraction.

Features in an 8-layer Model

Adam Jermyn, Tom Conerly, Trenton Bricken, Adly Templeton

In *Towards Monosemanticity,* we highlighted scaling dictionary learning to larger models as a key direction of fu work. We have since applied dictionary learning to the MLP activations of an 8-layer transformer, and identifiec number of interesting features (some examples below).

Broadly, what we see is that the features in the final layer look quite similar to the ones we saw in a 1-layer transformer. These fire in simple situations, often on single tokens, and predict plausible next tokens.

In earlier layers we see more abstract features, which activate on e.g. text with strong emotional valence. Very qualitatively, our sense is that layers 4-6 contain the most high-level features, though we have not done a caref census of these. This is consistent with the findings in *Softmax Linear Units* that the interpretable neurons in the middle layers of large models tend to fire on more abstract/high-level concepts than in early/late layers.

While an 8-layer transformer is still quite small compared with frontier models, it is a promising sign that we co to see interpretable features in models with more than one layer (consistent with the findings of Smith 2023 or layer transformer).

FEATURE EXAMPLES

These are some examples of the features we found. Note that we were preferentially looking for potentially saf relevant features, making this very much not a random sample of the features in this model.

Our confidence in these descriptions is moderate, though lower than for the features we inspected in detail in *Towards Monosemanticity*. We would encourage viewing these labels in the spirit of a biologist's field notes, rat than as rigorously established findings.

Layer	Features Found While Searching for Safety-Relevant Features	Directionally Safety-Relevant Features
2	Single-token "I" in prose about sex/romance.	Single-token 'with', preferentially in emotionally-sali contexts or describing ways someone shows emotio Single-token 'was' describing properties of who a pe is or what their mental/immediate physical state is.
3	Single-token ":" in news headings	Single-token "As" in violent/dramatic/emotionally charged scenes

4	Single-token "." demarcating a sharp transition in context. Activates on descriptions of countries taking actions.	Activates on causes of death. Activates in text with strong emotions, negative vale Single-token "called" spoken in anger/frustration. Single-token "him" in emotionally intense settings. Single-token for quotation marks in settings involvir strong negative valence, often political. Single-token newline in emotionally charged contex
5	Newline before another newline in historical contexts. Single-token "show" in the context of evidence. Single-token "comma" between an "if" clause and the "then" result (i.e. "if X, then Y" or "if X, do Y", etc.). Progressive political messages. Body language/embodied emotions.	Single-token "." in sentences expressing hatred. Single-token "?" in simple social reasoning question (questions of trust, confidence, identity, relevance, e
6	Descriptions of physical spaces. Sayings, quotes, and slogans. Sex/love/feelings in close relationships. "Of" after a group of two or three people (e.g. "two of them", "three of them"). Body language/embodied emotions. Nonverbal communication.	Strong negative emotions. Violence detector. Descriptions of interpersonal conflicts.
7	LaTeX math mode. Hebrew. Swedish. Repeated letters. Email spam. Base62.	Single-token "," in emotionally charged settings, predicting words indicating emotional state.

Multilingual Features in Large Models

Adly Templeton, Tom Conerly, Jonathan Marcus, Tom Henighan, Joshua Batson, Chris Olah, Adam Jermyn

As mentioned above, we've used dictionary learning to extract interpretable features from an 8 layer model. Th part of a broader effort to scale up dictionary learning, and we've been able to scale dictionary learning signific further -- out to models with billions of parameters.

We now have preliminary features in some of these models, and remarkably see that many of them are *multilingual*. That is, we see features that fire for the same concept across many different languages.

For example, we have found a feature that fires on the concept of uniqueness in many languages, including ph such as:

- “that’s the only way”
- “that’s the only hope”
- “is the only answer”
- “que es la única manera de que”
- “porque é a única forma”
- “Только таким аскетическим способом”
- “einzige sein die”

We have also found multilingual features firing on:

- Words meaning “two”
- Words meaning “sad”
- Quantifications of time
- The concept of whole/entire
- Questions starting with “why” or equivalent
- Descriptions of future needs (e.g. “will need to be developed”)
- Prose involving mathematics, especially in the context of taxation
- Official government statements (e.g. court documents & reporting on court proceedings)

Finally, we found some features which are not obviously multilingual but do seem interesting in their own right

- Expressing that someone is at their limits, that something is already too hard, etc.
- Calm down, lighten up, cool off, shut up, slow down, etc.
- Pointers to supporting evidence or causation (e.g. “as a basis for comparison”, “criteria for”, “the sole ba for”, “probable contributor to”, etc.)

Ghost Grads: An improvement on resampling

Adam Jermyn, Adly Templeton

Our views on ghost grads have changed. See the March 2024 update.

Dead neurons in the sparse autoencoder cost us compute and don't provide any benefit, so we'd like to make t live again. In our previous paper, we used a method we called "neuron resampling" to address dead neurons. S then, we've found a new method which works significantly better – Ghost Grads.

The method is to calculate an additional term that we add to the loss. This term is calculated by:

1. Computing the reconstruction residuals and the MSE loss as normal.
2. Computing a second forward pass of the autoencoder using just the dead neurons. In this forward pass replace the ReLU activation function on the dead neurons with an exponential activation function.
    1. We determine which neurons are dead for these purposes by applying a threshold to the numb samples since the neuron last activated. (Presently, we use a threshold of a neuron not activatin 2e6 tokens.)
1. Scaling the output of the dead neurons so that the L2 norm is ½ the L2 norm of the autoencoder resid from (1). Note that the scale factor is treated as a constant for gradient propagation purposes.
2. Computing the MSE between the autoencoder residual and the output from the dead neurons.
3. Rescaling that MSE to be equal in magnitude to the normal reconstruction loss from (1). The normal reconstruction loss is treated as a constant in this step for gradient propagation purposes.
4. Adding the result to the total loss.

This procedure is a little convoluted, but the intuition here is that we want to get a gradient signal that pushes parameters of the dead neurons in the direction of explaining the autoencoder residual, as that's a promising p parameter space to add more live neurons.

Note that in step (2) we use an exponential activation function because this has the joint properties of (a) being positive and (b) having a positive gradient with respect to the pre-activation. These mean that the model can o increase the magnitude of the dead neuron activations by making the pre-activations less negative. Combined our scaling in step (3), which ensures that the magnitude of the dead neuron outputs is smaller than that of the residual it is compared against, means that the model tends to receive a gradient signal to increase the pre-activations and hence make neurons alive.

Empirically we find that this procedure produces autoencoders with very few (often zero!) dead neurons. More we find that models trained with Ghost Grads perform as well or better than models trained with neuron resam at the same number of alive neurons.

Ghost Grads roughly doubles the compute requirements of the autoencoder, so it would be reasonable to ask cost is worth the benefit. Empirically, however, we find that the fraction of neurons that die increases with the s the autoencoder, and very large autoencoders can easily have more than 50% of neurons dead, even with neur resampling. So just on pure compute grounds, this means it is often advantageous to run a smaller autoencode Ghost Grads than to use a larger autoencoder with traditional neuron resampling.

Counterexamples in Superposition

Brian Chen, Chris Olah

In *Towards Monosemanticity*, we discussed a counterexample that caused us to deprioritize trying to make monosemantic models directly by regularizing the base model to have sparse activations. This counterexample the culmination of a series of thought experiments and real sparse models, and we discovered several other interesting phenomena and examples along the way.

In particular, we offer two surprising examples:

- An example of joint superposition between a neuron in an MLP layer and the residual stream running p to it.
- A neuron where different features are represented by different magnitudes (contrary to the linear representation hypothesis).

Both of these examples (as well as the example in *Towards Monosemanticity*) demonstrate phenomena that we to be cautious of in trying to produce monosemantic features. The last thing we want is an "out of the frying pa into the fire" situation where we trade superposition within a layer for something even harder to deal with!

Joint Superposition Between MLP Neurons and Residual Stream

It turns out that sparsely activated neurons can be “in superposition with the residual stream”.

In a 1-layer models we regularized to have sparse activations, we found a very sparsely activated neuron that primarily responds to three different types of inputs: "it|'s", "and|/", and "many| of" / "some| of". We confirmed for each of these facets, no other neuron activates to disambiguate the cases. At first glance, this seems like a polysemantic neuron without superposition, but this model’s output was able to disambiguate what the neuro represents by looking at it in conjunction with the residual stream and then use it to make reasonable predictic specific to each of the three facets.

In response to this obstacle, one might try to analyze or sparsify the residual stream instead. However, this is le natural since the residual stream does not have a privileged basis. To the extent that we want to understand M layers as a component of the overall model, superposition contained within the MLP layer is preferable to superposition between the MLP layer and the residual stream, so techniques that replace the former with the la are counterproductive for us.

It's worth noting that the phenomenon demonstrated by this example might be seen as a case of a more gene issue of "cross-layer superposition", which we expect to discuss more in the future.

Joint Superposition Between MLP Neurons and Residual Stream

Another obstacle is that, in the limit of forcing activations to be sparse, the model can use a “magnitude-based representation”. This directly opposes the linear representation hypothesis, and is more similar to a polytope representation.

To eliminate the superposition described above (at the cost of severely limiting model performance!), we traine another regularized 1-layer model with a “cut” residual stream: instead of adding the output of the MLP layer b the residual stream, we replaced the residual stream entirely with the output of the MLP layer. Even when only neuron fired on any given example, we still saw polysemanticity! In particular, we found one neuron firing weak one facet (personal pronouns) and strongly for a completely unrelated facet (the “m” of the word “diplomacy”), no other neuron activated to disambiguate the cases. Furthermore, we found that the one-layer model used th feature productively: when the neuron fired weakly, the model predicted tokens that make sense after persona pronouns, such as “ was”/” had”/” said”, and when it fired strongly, it predicted “acy”.

This is possible because the output layer has a softmax of a linear transformation. For example, a model could represent mutually exclusive, binary (on-or-off) features A, B, C in one neuron x by having $x = 1$ represent A, represent B, and $x = 3$ represent C, and then recover the feature in the output by computing $\mathrm{softmax}(x, 3x - 3, 5x - 8)$, or even $\mathrm{softmax}(100x, 300x - 300, 500x - 800)$ for higher confidence. W this type of representation is even harder to work with and reason about than superposition of linear represent so we expect techniques that encourage models to use it to be counterproductive as well.

Predicting Future Activations

Adly Templeton, Joshua Batson, Adam Jermyn, Chris Olah

A sparse autoencoder is trained to take in the transformer's activations at a specific location, such as after the nonlinearity in the MLP, and predict the same activations. These approximate the activations with a sparse representation. Instead, we could train a model to take in the activations at one point and predict the activatior some later point in the transformer. These approximate part of the network's computation as a sparse computa

We've been experimenting with three variations on this theme:

1. Take in the pre-nonlinearity MLP activations, and predict the post-nonlinearity MLP activations.
2. Take in the pre-MLP residual stream and predict the final output of the MLP layer.
3. Take in the pre-attention residual stream and predict the final output of the attention layer to the resid stream. For this method, we use a wide attention block with some sparsity regularization. We're current uncertain about what the best way to do this sparsity regularization is.

The first two variations do produce interpretable features, though we cannot yet confidently say whether these features are better or worse than our baseline sparse autoencoder. However, they have a major benefit in maki circuit analysis much easier, as the input weights of the sparse model compose directly with residual stream ele (and therefore outputs of earlier layers) and the output weights of the autoencoder compose directly with the residual stream elements (and therefore inputs to later layers), with the nonlinearity sandwiched in between. Th allows for an extension of the linear analysis in A Mathematical Framework for Transformer Circuits to include c supporting MLP features as well as output logits and attention heads.

We're currently in the very early stages of exploring the attention variation.

Random Open Problems

Adam Jermyn, Joshua Batson, Chris Olah

From time to time we notice open problems in interpretability that we would be excited for external researcher pursue, and which we either don't anticipate pursuing in the near future or which we expect to benefit from pa exploration.

Our intention is to have a low bar to posting problems in this section, so please treat these in the spirit of off-tl answers to "What are some interesting open problems in interpretability right now?"

With that in mind, we have listed two kinds of open problems below: ones oriented towards "Model Biology" (e going and looking inside models to see what they do) and ones oriented towards toy models.

MODEL BIOLOGY-ORIENTED PROBLEMS

1. Find surprising uses of attention in real models.
2. Mechanistically characterize even very simple attentional circuits, where each head is understood at the of "the QK circuit is selecting for things in class X and then the OV circuit is pulling information Y out of residual stream to produce outcome Z".

TOY MODELS

1. Produce a toy model, with a task and architecture analogous to some part of a large language model, v demonstrably learns to represent information *not* in superposition. The aim here is to produce alternati the superposition hypothesis.
2. Construct a monosemantic toy model, then compress that into a smaller model in superposition. Study kinds of errors this tends to produce, how the weights in the smaller model are related to those in the l one, etc.
3. Construct a small attention-only language model on a task that requires more working memory than w naively fit in the residual stream. Mechanistically analyze how this model makes use of the residual stre
4. Construct a toy language model which exhibits cross-layer superposition.

New Comment Digest

Transformer Circuits periodically publishes comments on our papers, both from external parties and by the autho Some of these comments were submitted before publication, from reviewers of early draft manuscripts. But others submitted significantly after the fact, and might not be seen. To that end, we've included a digest of recently adde comments:

Towards Monosemanticity: Decomposing Language Models With Dictionary Learning

- Replication & Tutorial (Neel Nanda)

Research By Other Groups

Joshua Batson, Nicholas L Turner

Finally, we'd like to highlight a selection of recent work by a number of researchers at other groups which we belie be of interest to you if you find our papers interesting.

MECHANISMS OF IN-CONTEXT LEARNING

Two concurrent papers – In-Context Learning Creates Task Vectors by Hendel *et al.* and Function Vectors in Larg Language Models by Todd *et al.* – addressed the question of how the tasks/functions leveraging in-context lea are represented. Hendel *et al.* study a series of in-context learning tasks of the form [A]→[B], where each of A a are single tokens, such as "Apple→Red Lime→Green Orange→", etc. They find that the value of the residual stre sitting at medium depth over the final "→" token contains information specifying the task, such that patching ir value in a fresh context "Lemon→" would produce the correct output ("Yellow" in this case). This indicates that task itself, as opposed to its answer for that example, gets encoded. They also check that the "task vector" is approximately the same independent of the 10 context examples used for extraction of the vector, indicating tl not merely a summary of the examples.

Todd *et al*. similarly study tasks of the form [A]:[B], such as "awake:asleep, vanish:appear, future:past", but they investigate which vectors get written to the final token ":" by which attention heads, rather than merely capturi residual stream vector at that position. They ablate the outputs of specific attention heads writing to the final " token, and identify a handful (~12 heads) which contribute significantly. They find that patching in those outpu a sufficiently early layer) restores the model performance on a fresh task context "night:" This confirms that the itself, as opposed to its answer, gets encoded. They work with multitoken entities, and demonstrate task arithm for a contrived series of tasks, such as "Copy-last-element-of-list + Give-capital-city-of-first-element-in-list - Co last-element-of-list" which does produce "Give-capital-city-of-last-element."

We would be excited to see follow-up work checking if the same mechanisms are involved when the task to be performed is determined by context and standard usage, as opposed to an in-context-learning sequence of analogies. For example, when the model processes the sentence "The color of an Orange is", does the same "ta vector" (whether captured from residual stream of summed from attention head outputs) form on "is" as forms "→" in "Apple→Red Lime→Green Orange→"? If so, then ICL examples could serve as "purifying" strategies for identifying model mechanisms that are used more generally.

INDUCTIVE BIASES

Given a specific prediction problem with multiple possible exact and approximate algorithms, which ones will a neural net learn? How does that depend on architecture, regularization, and training scheme? While mechanist interpretability provides techniques for reverse engineering a particular model, any traction we can get on wha algorithms to expect provides a very useful starting point for any analysis, and for reasoning about out-of-distribution behavior.

While the general problem on natural language or image datasets is quite difficult, there have been a number o recent suggestive results in the context of small, purely algorithmic tasks.

In Allen-Zhu & Li, the authors show that transformers can learn to generate Context-Free Grammars (strings generated by a series of replacement rules). Classical algorithms solve this problem with dynamic programmin the authors show using probes that transformer hidden states linearly encode the relevant internal states of th dynamic programming solutions ("ancestors and boundaries") and store them at ends of phrases (similar to punctuation in LMs).

Singh et al. shows that for a task which can be solved via in-context learning or in-weights learning, the favore algorithm depends both on training time and regularization strength. Concretely, they train an autoregressive on a sequence of pairs of (drawn glyph, label); ICL would learn the glyph shape → label map from examples in context, while in-weights learning would learn a fixed set of those relationships from training. While the author investigate the two algorithms mechanistically, the former might involve doing a sort of fuzzy induction head (some representation of the drawn glyphs for keys and queries). They find that turning on L2 weight regularizat strongly favors ICL solutions, as in grokking, while overtraining in the absence of regularization eventually favo weights solutions. They also provide evidence suggesting that these two algorithms compete for space within residual stream.

Zhong et al. returns to the task of modular addition, showing that a 1-layer transformer with constant attentio learns an algorithm ("pizza") which is qualitatively different than the algorithm ("clock") learned by a 1-layer transformer with ordinary attention investigated in Nanda *et al.* Strikingly, both these algorithms learn fourier/ embeddings of the one-hot encoded numbers. Morwani *et al.* proved (!) that any maximum margin solution to modular addition (in a 1-hidden layer MLP with quadratic activations) must have such Fourier features, and tha optimal performance requires that all Fourier frequencies be present. In fact they show something much more general. Chughtai *et al.* had observed that models trained on finite group multiplication learn an algorithm bas the characters of finite-dimensional group representations, and Morwani *et al.* show that optimal performance requires a model to learn all of the group's finite-dimensional irreducible representations. We believe this mark first time that an algorithm discovered by reverse engineering a neural network was later proved to be mathematically optimal. (The reverse has happened, e.g. ridge regression is optimal for noisy least-squares pro and is learned by transformer models (Akyürek *et al.*))

What about more general classes of algorithms? The programming language RASP(-L) attempts to abstract the algorithmic capacity of transformers (certain kinds of lookups and aggregations). Zhou *et al.* demonstrate thro examples that problems with variable length inputs that can be solved by short RASP-L programs (independen input length) are easily learned by transformers, in a way that generalizes over length. For example, it is hard t implement traditional addition (largest digit first) with a short RASP-L program, because you have to get to the of both numbers to get started and do complex indexing. If the addition problem has index hints and reverses order, it is easy to write in RASP-L and empirically transformers learn length-independent solutions. They conje that any problem with a short RASP-L program will be easy to learn for transformers. (The converse won't hold, certain dense computations or analogic reasoning tasks are not amenable to RASP representations but can be learned by transformers.) One might hope that something deeper is true: there is some notion of 'transformer-complexity' for algorithms (for which RASP-L is an upper bound) and that the algorithms learned are some complexity-weighted distribution over those, as in Solomonoff's Universal Induction.

PROBING AND CIRCUIT DISCOVERY

In Fact Finding: Attempting to Reverse-Engineer Factual Recall on the Neuron Level, Nanda *et al.* investigate th circuit supporting a specific narrow case of factual recall: completions of the sentence "Fact: [ATHLETE NAME] p the sport of" for a collection of [ATHLETE NAME]s playing basketball, baseball, or tennis. They find that detokenization happens in the first layers (0, 1), factual recall happens in the early MLP layers (2-6), after which attention heads (primarily one specific attention head) selects the sport information and transmits it to the fina token. The attention part of this story is tidy – composing the OV circuit with the unembedding matrix even pro a mechanistic probe for sport. But the factual recall MLP story is not; the authors carefully falsify a few simple th of how the factual information about entities might be stored, and speculate about the limits of the quality of explanation we might expect for such unstructured and arbitrary facts about entities. While we are optimistic th there are principles behind how networks store even arbitrary information, it is as yet unclear what the minimal interpretable unit will be, especially when many layers are (provably) involved.

SUPERPOSITION AND DICTIONARY LEARNING

We're excited to see our enthusiasm for using dictionary learning to attack superposition (along with Cunningh al.) spread within the broader community. In Sparse Autoencoders Work on Attention Layer Outputs, Kissane *e al.* use sparse autoencoders to extract interpretable structure from the attention-weighted value vectors (or "*ho* vectors in their ecosystem) within the second attention layer or a two-layer transformer. They find activation specificity for three families of features within this vector space. Induction features activate when a particular to should be predicted following the logic of an induction head, and link these to the induction heads of this atte layer. Local context features activate within short-term contexts that have a clear delimiter (e.g., something akir this clause a question?"). Finally, high-level context features describe something like the document-level contex likely specified by certain keywords within a context.